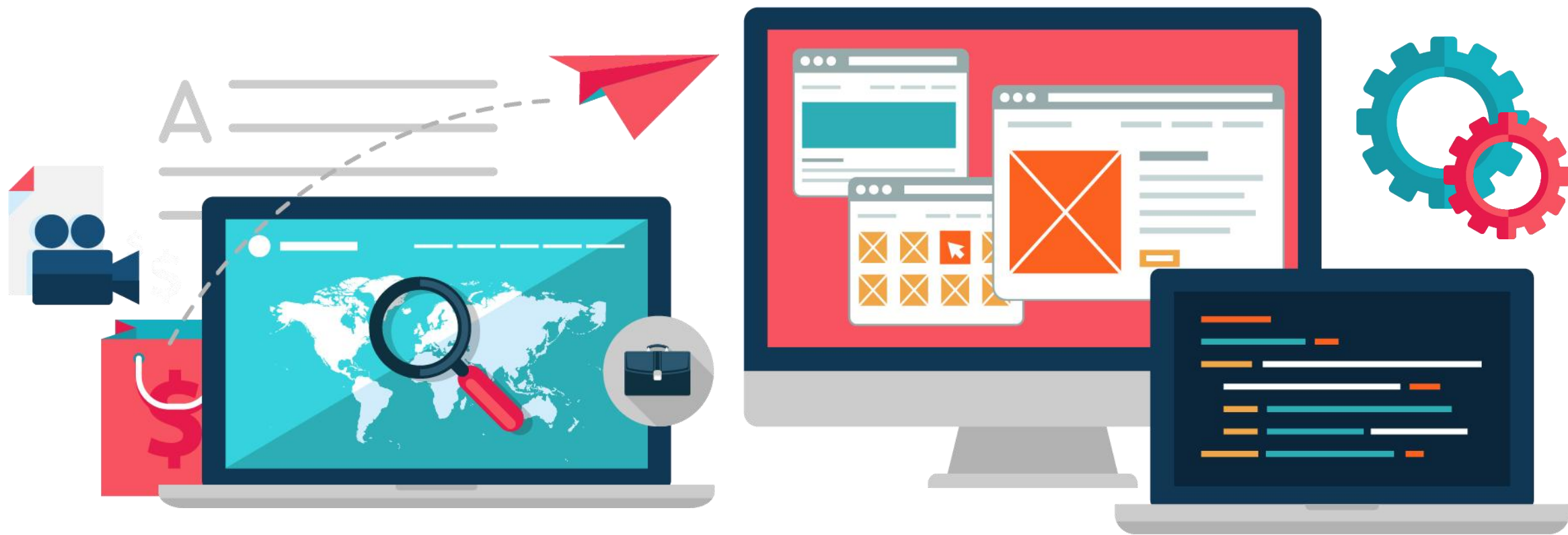


面向对象程序设计

Object Oriented Programming





指针、数组、引用、常量



01

指针

02

数组

03

引用

04

常量

05

const和指针

06

const和引用



◆ 格式: `Type * varName [= 地址表达式];`

◆ 例:

```
int    num    = 5;  
int *  pNum   = &num;  
int *  pN2    = pNum;  
void   *      p = 0;  
bool   **     p = 0;  
int    **     ppNum = &pNum;
```

指针在定义的同时**应当**初始化

```
int *  pName = NULL; (C++98) //f(int); f(char*); f(NULL);  
int *  pName = nullptr; (C++11)
```



◆ 指针初始化和赋值操作的约束

1. 0值常量表达式, NULL或nullptr

例: `int *pi=0;` 或 `const int num = 0; int *pi=num;`

2. 类型匹配的对象的地址(除void*外)

例: `double dval; int *pi=0; pi=&dval; //错误`

3. 同类型的某个对象后面的另一对象

例: `int a[2]={1,2}; int * p1=&a[0]; int * p2=++p1;`

4. 同类型的另一个已正确初始化的有效指针

例: `int *pd; int *pi=pd; //错误`



◆ 未初始化的指针是无效的，直到给该指针赋值后，才可使用它

```
char *pch;    int * pin;  
cout << pch;  //运行时异常， “内存不能读”  
cout << pin;   //输出 “CCCCCCCCC”  
cout << *pin;  //运行时异常， “内存不能读”  
*pin = 10;     //运行时异常， “内存不能写”  
pch = "Hello World" ; //C++11之前考虑兼容允许但不推荐  
cout << pch;   //C++11之前运行时正常  
int num(0); pin = &num;  
cout << *pin;  //运行时正常  
*pin = 10;     //运行时正常
```



- ◆ 指针必须初始化或赋值成为有效指针后才能使用。C++编译器无法检测指针是否未被初始化，也无法区分有效地址和非法地址；如果使用未初始化和未赋值的指针，将会访问到未知的内存，有可能带来程序崩溃
- ◆ 定义指针时如果不能确定其所指向的对象，则先将指针初始化为 0 或 `nullptr`。因为编译器可检测出 0 值的指针，程序可判断该指针并未指向一个对象
- ◆ 当给指针初始化或赋值的时候，应当提供给它一个对象地址值，而不是一个对象值

◆ dereference: 解引用、间接引用、逆向引用

◆ 例: `int a= 8; int * pa = &a; int ** ppa = &pa;`

```
int b =9;  
*pa = b;  
*ppa = &b;
```

◆ 指针的运算:

例: `int a[2] = { 1, 2 }; int * p = &a[0];`
`cout<<*(++p);`

`T * p = ....;`

则 `p+n` 等价于 `(T*) ( ((char *) p + sizeof(T) * n) )`

取数组元素 `p[n]` 就是 `* (p+n)`



◆ 格式: `Type varName[ constexp(可选) ] [= {初值列表}];`

例 (C++98):

```
int a[5] = {1,2,3,4,5}; int* b=a; int* c=&a[0];
```

例 (C++11新增):

```
int a[5] {1,2,3,4,5}; int* b=a; int* c=&a[0];
```

◆ 说明:

`Type name[ constexp ]` → `Type[ constexp] name`

例: `int (*p[4])(int ,int)` → `int (*)(int ,int) p[4]`



- ◆ 数组在定义的同时**可以**显式初始化，即为其元素提供一组用逗号分隔的初值，这些初值用花括号{}括起来，称为初始化列表

```
int a[3] = {1, 2, 3};
int b[ ] = {4, 5, 6}; //此时数组不需要指定大小
char c[2] = { 'a' , 'b' , '\0' }; //错误，初值太多，越界
int c[5] = {1, 2, 3}; //初值太少，剩余元素被赋为0
char e[5] = "hello" ; //错误，字符串字面值结尾有\0
a[3]=4; //错误，数组越界，visualstudio编译无错误，运行异常
int d=20; //codeblock输出可能为20或其他，越界取值不可预测
cout << a[3];
```
- ◆ 数组不能用另外一个数组初始化或赋值

```
a = b; // fail
```

◆ 多维数组格式:

(C++98) Type varName[constexp1][constexp2]... [= {初值列表}];

(C++11) Type varName[constexp1][constexp2]... [{初值列表}];

◆ 指针数组与数组指针

指针数组:

int * array[3];

数组指针:

int (*p)[3];

◆ 数组引用

void g(int (&ra)[3]);

例:

int a[4] = {1,2,3,4};

int * b[4] = {&a[0],&a[1],&a[2],&a[3]};

int (*p)[4] = &a;

cout << (*p)[3] << endl;

p++ 偏移多少字节?



- ◆ C++ 数组名可用作到它第一个元素的指针

```
int a[4] = {1,2,3,4};
```

```
int *pa = a; int *pa = &a[0];
```

```
cout<<p[3];
```

- ◆ C++ 数组名字**可以看作**一个指针常量，其指向数组的第一个元素

- ◆ 可以通过指针访问数组中的每个元素

一维数组的指针表示法：

例：int a[4]; 用a[i] or *(a+i) 访问数组元素，i取0 1 2 3

二维数组的指针表示法：

例：int b[2][5];

b[i][j] or (*(b+i)+j) or *(b[i]+j) or (*(b+i))[j]
or *(&b[0][0]+5*i+j)

◆格式： <类型> & <变量> = <对象或变量>; //同时初始化

例如： `int a = 10; int & b = a;`
`cout << a << b << endl;` //输出10和10
`a = 6;`
`cout << a << b << endl;` //输出6和6
`b = 5;`
`cout << a << b << endl;` //输出5和5
`&a == &b ?`

◆引用的含义

- 是一个别名
- 必须初始化(存在例外)
- 对应的变量/对象，必须存在



◆引用的定义及初始化

例如: `int num=0;    int &r=num;    // 正确`
 `int &r;                                // 错误`
 `double d(3.14); int &r=d;    // 错误`
 `int &r=10;                                // 错误`
 `r = 10;                                // 相当于把num的指修改为10`
 `int &r1=num, &r2=num;    //一行中定义多个引用`

◆引用定义的同时必须用类型匹配的对象初始化

◆引用初始化后, 只要该引用存在, 它就保持绑定到初始化时指向的对象, 不可能将它绑定到另一个对象



//使用变量a和b

```
int a =7,b=8;
```

```
void f(int a,int b) { cout<<a<<b<<endl; }
```

调用方: **f(a, b);**

//传指针, 修改变量a和b

```
int a =7,b=8;
```

```
void f(int *pa,int *pb) { *pa=1;*pb=2; }
```

调用方: **f(&a, &b);**

//传引用, 修改变量a和b

```
int a =7,b=8;
```

```
void f(int &a,int &b) { a=1;b=2; }
```

调用方: **f(a, b);**



◆ 引用和指针之间的共同点

1. 都可以作为函数的参数和返回值
2. 都是左值，可以放在赋值运算符左侧

◆ 引用和指针之间的区别

1. 引用通过别名直接访问，指针通过地址间接访问
2. 引用在定义时必须初始化。可避免使用无效指针
3. 引用在初始化后不可改变。指针可用于遍历数组元素
4. 不存在空引用。当所指对象可能不存在时可用指针

```
int fun(int a) { return a; } int &r = fun(10); //错误
```



- **文字常量(字面常量)**

100, 3.14, true, 'C', "吉林大学" ,..

- **宏定义**

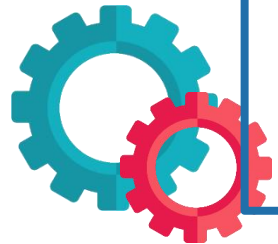
```
#define VALUE 98
```

- **命名常量**

```
const int CARD_COUNT = 54;
```

- **const和指针**

- **const和引用**



//这段代码好理解吗？易维护吗？

```
if (var==1) {  
    value = 3.14*2.5*2.5;  
}  
else if(var ==2) {  
    value = 3.14*2.5*2.5*2.5*4/3;  
}  
else {  
    value = 2.5*2.5;  
}
```




◆ 缺点

◆ 宏定义只是进行简单的字符替换，没有类型安全检查

```
#define PI 3.14    char name[20];    strcpy(name, PI);
```

◆ 宏定义还会产生边际效应

```
#define N (2+3)
```

```
#define MIN(a, b) (a > b ? b : a)
```

◆ 无法对宏定义的符号常量进行调试

◆ 宏定义的符号常量的作用域，从定义点到整个程序的结束点，容易与其它变量产生命名冲突



◆ 优点（有其它作用）

- 可用于条件编译，**#define XXX #ifdef XXX #endif**
- 可用于字符及字符串，**#, ##, #@**
- 编译器内置宏，能输出非常有用的调试信息

//条件编译(例)

```
#define DEBUG
```

```
int main( ) {  
    int a=100;  
  
    #ifdef DEBUG  
        cout<<a<<endl;  
        Do1( );  
  
    #else  
        Do2( );  
  
    #endif  
}
```

// #, ##, #@

```
#define STR(x) #x
```

```
#define VAR(x) n##x
```

```
#define TOCHAR(x) #@x
```

```
int n1=1,n2=4,n3=7;
```

```
cout<<STR(123)<<endl; //输出字符串 "123"
```

```
cout<<STR(abc)<<endl; //输出字符串 "abc"
```

```
cout<<VAR(2)<<endl; //输出变量n2的值4
```

```
cout<<TOCHAR(2)<<endl; //输出字符 ' 2'
```

```
cout<<TOCHAR(a)<<endl; //输出字符 ' a'
```

//LINE、FILE、FUNCTION

```
cout<<__LINE__<<endl;
```

```
cout<<__FILE__<<endl;
```

```
cout<<__FUNCTION__<<endl;
```



- 格式:

```
const int CARD_COUNT = 54;
```

- 枚举量(C++98):

```
enum Color {  
    RED=0xFF0000,  
    GREEN=0x00FF00,  
    BLUE=0x0000FF  
};
```

- 枚举类(C++11):

```
enum class Color {  
    RED=0xFF0000,  
    GREEN=0x00FF00,  
    BLUE=0x0000FF  
};
```

枚举类型的枚举成员都是const对象，一旦定义就不能修改



◆ C++中推荐通过const来定义常量

◆ 格式

const <类型> <常量名> = <表达式>;

例: const float PI;

PI=3.1415926; //错误

const float PI=3.1415926; //正确

PI=3.1416; //错误

◆ const对象定义的同时必须初始化

◆ const对象定义后就不能被修改



- ◆ `const`对象与非`const`对象
- ◆ 把普通对象声明为`const`会改变它的类型。类型为`const int`的对象与类型为`int`的对象完全不同
- ◆ 在文件作用域定义的普通非`const`对象默认为`extern`，而`const`对象默认为非`extern`，即`const`对象默认为文件的局部变量，要使`const`对象能够在其他的文件中访问，必须显式地指定它为`extern`
- ◆ 在文件作用域定义的普通非`const`对象只能出现在某个源文件中，而`const`对象可以定义在头文件中，每个包含该头文件的源文件都有了自己的`const`对象，其名称和值都一样



- ◆ 如果是用常量表达式（编译时能够计算出结果的表达式）对 const 对象进行初始化，那么大部分的编译器在编译时都会将常量表达式计算求值，并用求得的值来替换对这些 const 对象的使用，并将其保存到符号表，而不会为其分配任何存储空间，这就是**常量折叠**(Constant Folding)。效果与宏替换一样
- ◆ 常量折叠是一种被很多现代编译器使用的编译器最优化技术，它在编译时间简化常量表达，提高了效率，节省了空间
- ◆ 宏是符号常量，宏替换后，编译器不再维护这个符号
- ◆ 常量折叠后，常量名并不会消失，编译器会把它放到符号表中，同时，也会根据需要为其分配空间



```
//const int i=0;  
int main()  
{  
    const int i=0;  
    int *j = (int *) &i;  
    *j=1;  
    cout<<&i<<" "<<j<<endl;  
    cout<<i<<" "<<*j<<endl;  
    return 0;  
}
```

运行结果:
0012ff7c 0012ff7c
0 1

说明i是可折叠常量，在编译阶段对i的使用已经被替换为i的值；同时i确实分配了空间。



- ◆ 用常量表达式初始化的const对象一般都可以定义在头文件中。这样，无论该const对象何时使用，编译器都能够引用其初始值，并能保证多个文件引用的是相同的值
- ◆ 不是用常量表达式初始化的const对象一般都不应该定义在头文件中。相反，和普通的非const对象一样，该const对象应该在某个源文件中定义和初始化，并在头文件中为它添加extern声明，以使其能被多个文件共享

```
//xx.h  
const int v1=100;  
const int v2=200;
```

```
//a.cpp  
#include "xx.h"  
...
```

```
//b.cpp  
#include "xx.h"  
...
```

两种形式等价

```
const int* pt = &a;  
int const* pt = &a;
```

例:

```
int v1 = 100;  
int v2 = 3;  
const int * p1 = &v1; //正确  
*p1 = 10; //错误  
cout << *p1 << endl; //输出100  
const int * p2 = &v2; //正确  
int * p3 = &v1; //正确  
int * p4 = p1; //错误  
int v3 = *p1; //正确
```

```
const int cv1 = 200;  
const int cv2 = 4;  
const int * pc1 = &cv1; //正确  
cout << *pc1 << endl; //输出200
```

```
int * pc2 = &cv2; //错误
```

```
int * p1 = pc1; //错误  
int v1 = cv1; //正确  
int v2 = *pc1; //正确
```



◆ 普通指针:

```
T * p [ = exp];
```

◆ 指向常量的普通(non-const) 指针:

```
const T * pt [ =exp ];
```

◆ 指向变量的常(const)指针:

```
T * const pt = exp; ( 必须初始化)
```

◆ 指向常量的常(const)指针:

```
const T * const pt = exp; ( 必须初始化)
```

```
T var = exp;
```

const

T

*

const

pt

=

&var;

&var

pt

exp

var

const

: 说明pt指针变量存放的值(&var)不能改变,即不允许pt指向其它地址。

const

: 说明禁止通过pt指针来修改其所指向的内容(exp)。但不限制其它指针修改var的内容。

```
int var1 = 5; int var2 = 10;
int * const pt1 = &var1; //正确
pt1 = &var2; //错误, 不允许修改pt1存放的值。
*pt1 = var2; //正确, 虽不能指向其它变量, 但能修改指向的内容
const int * pt2 = &var1;
*pt2 = var2; //错误, 不能企图修改*pt2
pt2 = &var2; //正确, pt2可以指向其它量
var2 = 99; //正确, 只要不通过pt2指针, var2可任意修改。
cout << *pt2; //这时*pt2的值是99.
```



const和指针 (表)

	int v=4; int m=7;	const int v=4; int m=7;	int v=4; const int m=7;	const int v=4; const int m=7;
int * pv = &v;	√	×	√	×
const int * pv2 = &v;	√	√	√	√
int * const pv3 = &v;	√	×	√	×
const int * const pv4 = &v;	√	√	√	√
pv= &m;	√	/	?	/
*pv = m;	√	/	?	/
pv2 = &m;	√	?	?	?
*pv2 = m;	×	?	?	?
pv3 = &m;	×	/	?	/
*pv3 = m;	√	/	?	/
pv4 = &m;	×	?	?	?
*pv4 = m;	×	?	?	?

◆ char * 和 char[]定义字符串

- `char * str = "This is a string!";` // vs fail, codeblock ok

等价于 `const char * str = "This is a string!";`

所以, `str[0] = 't';` //codeblock中运行异常

- `char str[] = "This is a string!";` //str[0] = 't'; ok

◆ 标准库中的string

```
#include <string>
```

```
using namespace std;
```



◆ 变量的引用

```
int val = 100;  
int & myval1 = val;
```

```
const int & myval2 = val;  
myval1 = 300;  
myval2 = 100;
```

◆ 常量的引用

```
const int b = 100; int& aa = b;  
const int& aa = b;  
const int& bb = 1;  
aa = 10;  
bb = 20;
```

```
int var = 100;  
const int cvar = 200;
```

```
void f(int a1, int &a2, const int& a3)  
{ }
```

```
f(1, 2, 3);  
f(1, var, var);  
f(1, cvar, cvar);
```

```
f(1, var, cvar);  
f(1, cvar, var);
```

```
f(var, var, var);  
f(cvar, var, cvar);
```



本章结束